

Low Level Design

Railway Reservation System

1. System Overview

The Railway Reservation System is a RESTful API designed to handle core railway operations. It allows users to search trains, check route-based fares using station codes, reserve tickets (including multi-passenger booking up to 6 tickets per request), perform online check-in with seat assignment, and provides administrative tools for managing train and route data.

Key Features Include:

- Secure authentication and authorization using JWT and role-based access (Admin and Passenger)
- Train search and station-to-station fare planning functionality
- Ticket booking with route validation, direction checks, and deferred seat assignment model

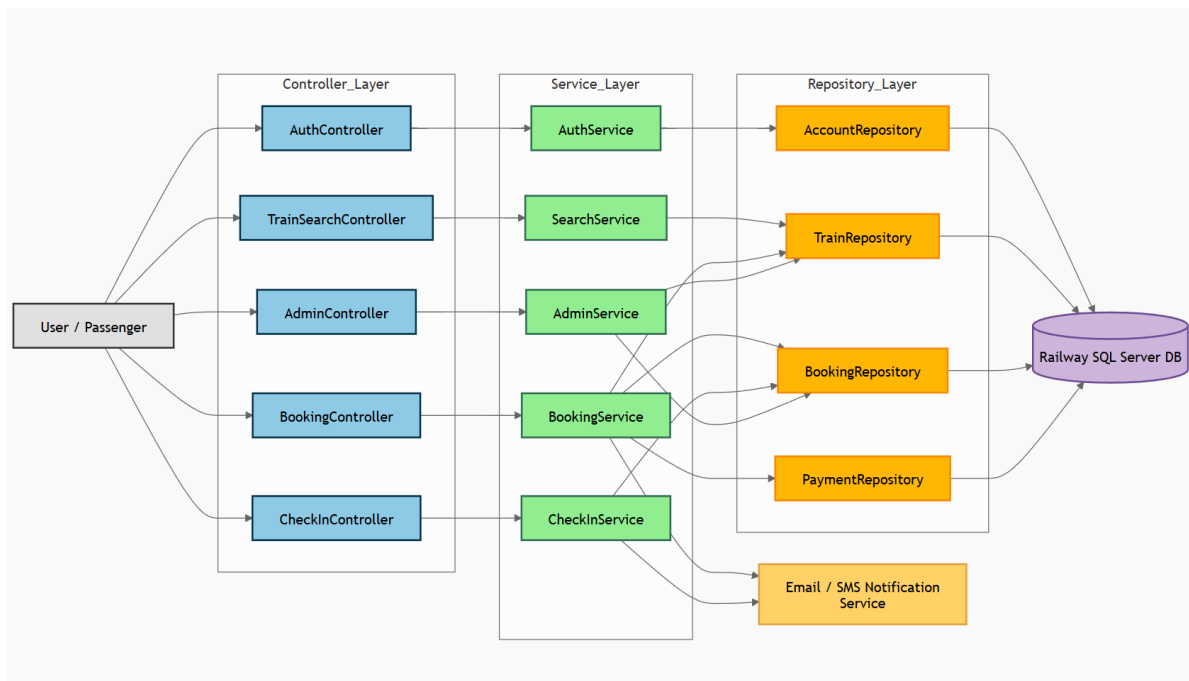


Figure 1 – Three-Tier Architecture Overview

Presentation Layer

This is the **UI-facing layer** that handles HTTP requests and responses for Railway Reservation System.

Components:

- AuthController
- SearchController
- BookingController
- CheckInController
- AdminController

Responsibilities:

- Receive user input via HTTP endpoints.
- Perform basic request checks (required fields, body format)
- Call appropriate repository methods.
- Return JSON responses with proper HTTP status codes

Business Layer

This layer contains all core railway domain logic and workflow decisions:

Components:

- AuthService
- SearchService
- BookingService
- CheckInService
- AdminService

Responsibilities:

- Enforce booking rules (max 6 passengers per request).
- Validate station-code travel direction and route consistency
- Calculate segment fare from route stations
- Handle deferred seat assignment at check-in

Data Layer

This is the **persistent storage layer**.

Component:

- SQL Server Database
- ApplicationDbContext (EF Core)
- Repository interfaces and implementations:
- AccountRepository
- TrainRepository
- BookingRepository

Responsibilities:

- Store and retrieve domain data (users, trains, route stations, bookings, check-ins).
- Execute EF Core queries and updates.
- Enforce indexes/constraints through schema and model configuration.
- Provide clean data access contracts to services.

2. Technology Stack

The backend is built using **ASP.NET Core 9.0** on **.NET 9**.

The project uses **Entity Framework Core 9.0** as the ORM, with **SQL Server** as the persistent database.

Security and testing tools:

- **JWT Bearer Authentication** is used for secure login and role-based authorization.
- **NUnit** is used for unit testing.

API documentation:

- **Swagger (Swashbuckle)** is integrated for interactive API documentation and endpoint testing.

Architecture style:

The system follows a **3-tier layered architecture**:

- **Controllers** handle HTTP requests and responses.
- **Services** contain business logic and workflow rules.
- **Repositories** handle data-access operations.
- **SQL Server** stores persistent application data.

Design patterns and practices used:

- Repository Pattern
- Dependency Injection (DI)
- DTO-based request/response contracts
- Middleware pipeline for cross-cutting concerns (exception handling, authentication, authorization)

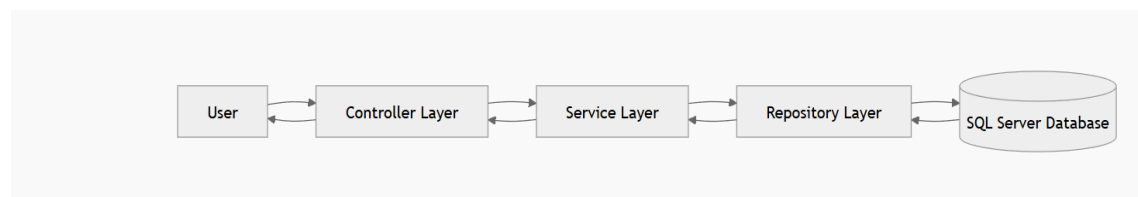


Figure 2 – Detailed Architecture: Controller → Service → Repository → DB

3. Project Structure

The application is organized into clearly defined folders:

- **Controllers:** Handle API endpoints for authentication, search, booking, check-in, and admin operations.
- **Services:** Contain business logic such as fare calculation, route validation, booking workflow, check-in seat assignment, and admin operations.
- **Repositories:** Define interfaces and implementations for data-access operations (account, train, booking) and isolate EF Core queries from business logic.
- **Interfaces:** Store contracts for services and repositories, enabling dependency injection and testability.
- **Models:**
 - Entities** represent domain objects such as ApplicationUser, Train, TrainStation, Booking, and CheckIn.
 - DTOs** represent request and response models used by APIs.
- **Data:** Contains ApplicationDbContext for database interactions and model configuration.
- **Middleware:** Includes a global exception handler for consistent API error responses.

Use Case Diagram

Actors (Users)

- **Admin**
- **Customer (Passenger)**

Admin Use Cases

- **Login**
- **Add / update / delete train and route data**
- **View train-level dashboard metrics (capacity, bookings, revenue)**

Customer Use Cases

- **Register and login**
- **Search trains and fares**
- **Book tickets**
- **Check booking/check-in status**
- **Perform check-in for booked tickets**
- **Cancel booked tickets**

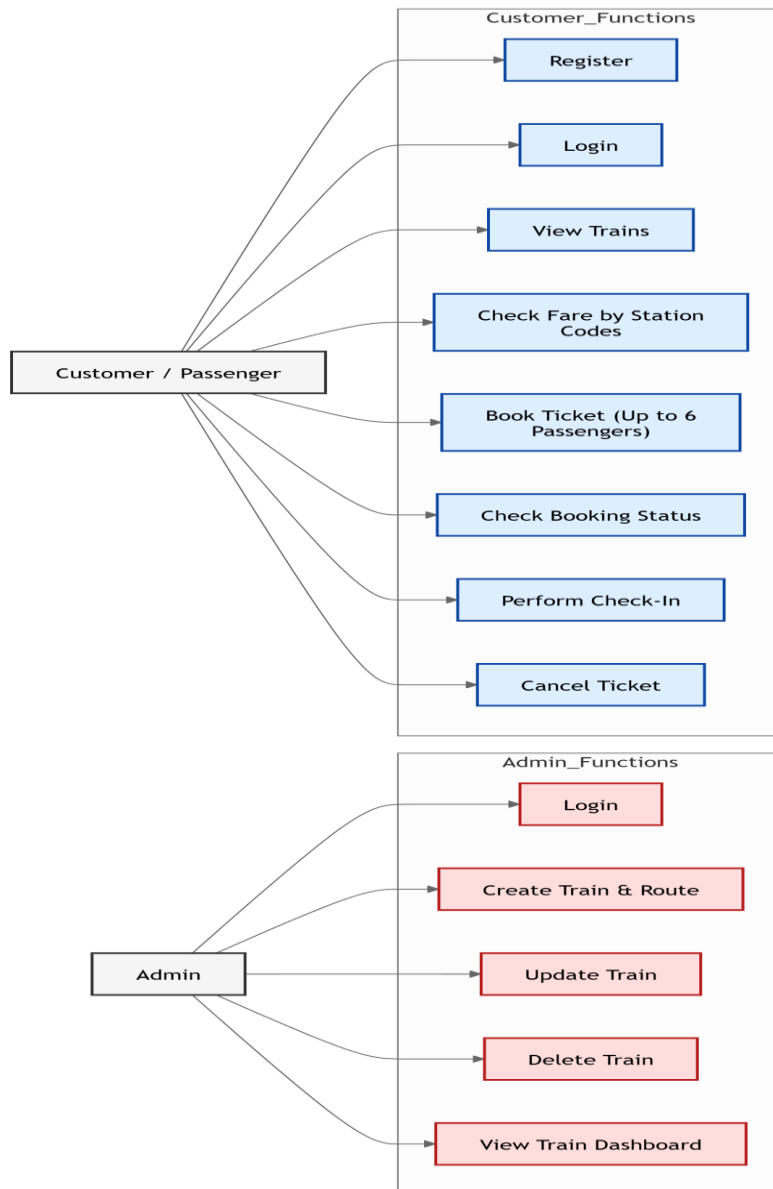


Figure 3 – Use Case Diagram (Customer & Admin Actors)

4.Database Design

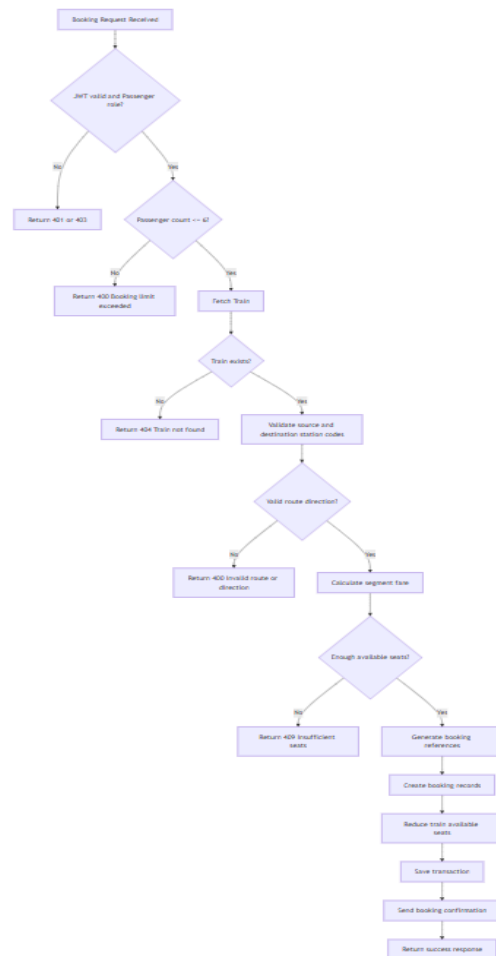
The system uses the following main tables:

- **AspNetUsers Table:** Stores user account details such as username, email, password hash, and full name through ASP.NET Identity.
- **Trains Table:** Contains train details such as train number, train name, source, destination, total seats, available seats, fare, and departure time.

- **TrainStations Table:** Stores route stop information for each train, including station code, station name, stop order, and cumulative fare from the starting station.
- **Bookings Table:** Tracks reservations using PNR, passenger details, train association, route segment, fare, booking timestamp, status, and check-in state. Seat numbers are assigned later during check-in.
- **CheckIns Table:** Stores check-in information for passengers, including booking reference, seat number, and check-in reference.

Indexes: A unique index is applied to the booking reference number, and a standard index is placed on the flight ID for efficient lookup.

DataFlow Diagram



When a user interacts with the Railway Reservation System, whether for login, train search, ticket booking, check-in, cancellation, or admin operations, the request first reaches the appropriate controller. For example, login requests go to AuthController, booking requests go to BookingController, check-in requests go to CheckInController, and train administration requests go to AdminController.

Each controller then forwards the request to a service layer. The service layer contains the actual business logic and domain rules of the railway system. This includes tasks such as validating station codes, checking seat availability, calculating route-based fare, generating PNR values, assigning seats during check-in, and updating booking status during cancellation.

After making business decisions, the service calls the required repository. The repository layer is responsible only for data access. It retrieves, inserts, updates, or deletes records using Entity Framework Core and SQL Server.

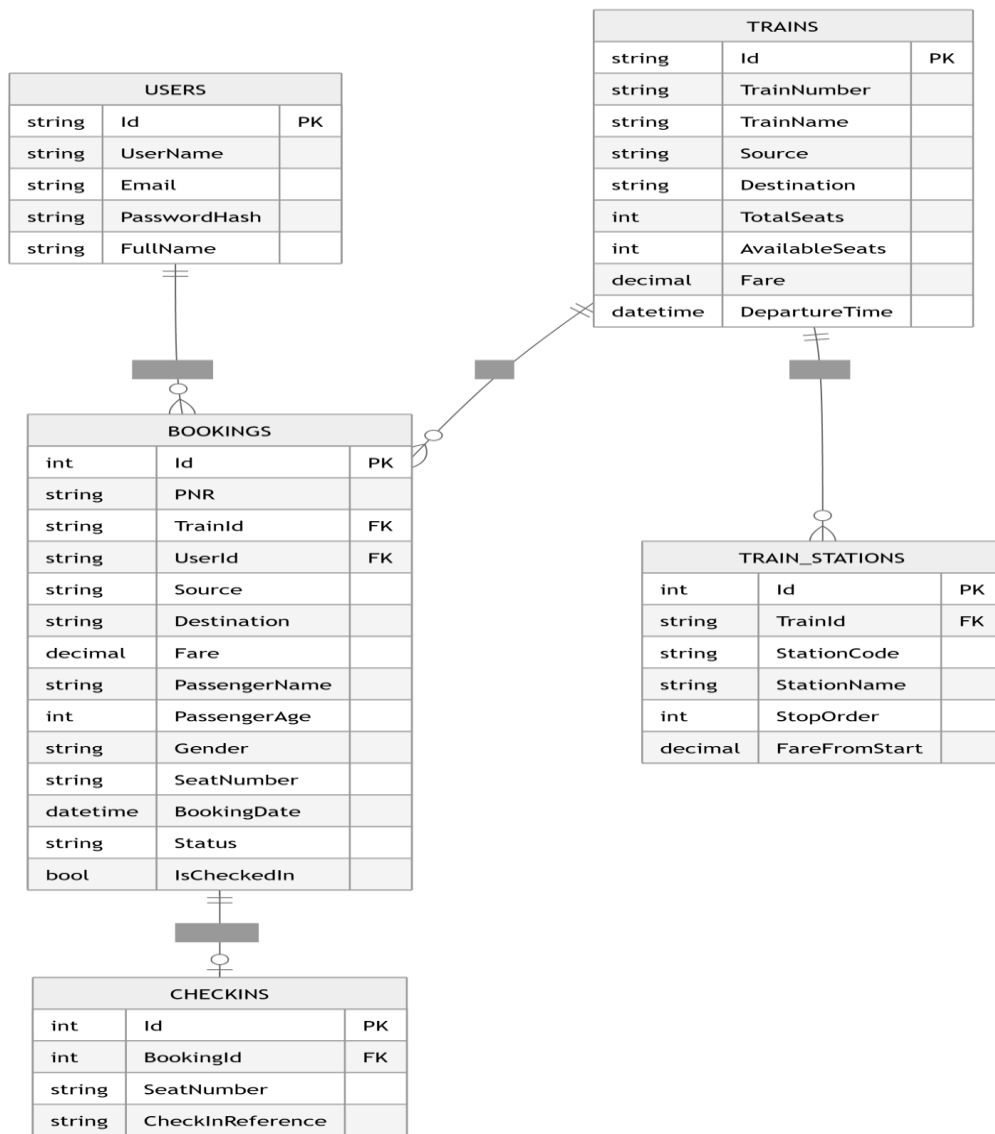
This layered flow keeps the code clean and maintainable. It reduces coupling between controllers and the database, improves testability, and makes the system easier to extend when adding new railway features such as coach allocation, waitlist handling, or timetable-based seat planning.

DataTable Diagram

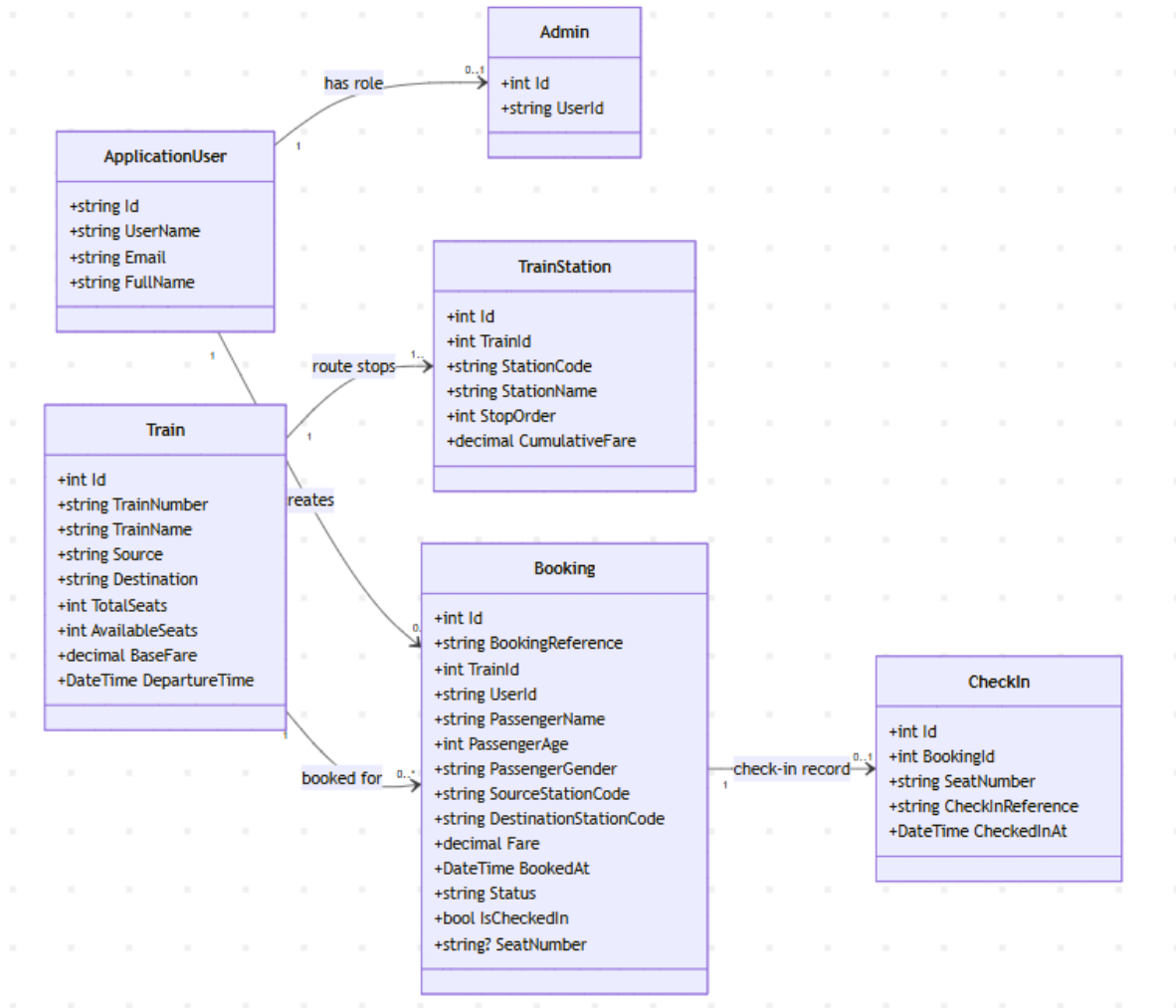
The system has five main functional tables: **Users**, **Trains**, **TrainStations**, **Bookings**, and **CheckIns**.

- The **Users** table stores passenger account details such as user identity, username, email, and profile information. Each user can create multiple bookings in the system.
- The **Trains** table stores master train information such as train number, train name, source, destination, total seat capacity, available seats, fare, and departure time.
- The **TrainStations** table stores the route information for each train. It contains station codes, station names, stop order, and fare from the starting point. This table is important for calculating station-to-station ticket fare.
- The **Bookings** table connects passengers to trains and stores reservation details such as PNR, passenger details, source station, destination station, fare, booking date, status, and check-in state.

- The **CheckIns** table stores the final check-in information for a booking, including assigned seat number and check-in reference.



5. Domain Models



The system defines core domain entities:

- **Admin:**

This entity represents system administrators. Admin users are responsible for managing train data, updating route-related information, deleting trains, and monitoring booking activity through the dashboard. In the current system, admin access is controlled through role-based authentication.

- **User:**
- This entity stores passenger account and login details. It includes identity-related fields such as unique user ID, username, email, and full name. Users can register, log in, search trains, reserve tickets, perform check-in, and cancel bookings.
- **Train:**

This entity contains flight-related information. It includes a flight ID, flight number (up to 10 characters), origin and destination cities (up to 20 characters each), flight date, and fare. Each flight can be booked by multiple customers.

- **Train Station:**

This entity stores the route details for a train. It includes station code, station name, stop order, and cumulative fare from the starting station. It is used to support intermediate-station travel, station-code validation, and route-based fare calculation.

- **Booking**

This entity captures reservation details. It includes booking ID, PNR, train reference, user reference, source station, destination station, fare, passenger details, booking date, status, seat number, and check-in state. A booking is created when a passenger reserves a ticket, and it remains confirmed even before seat assignment.

- **Check In**

This entity stores check-in information for passengers. It includes check-in ID, booking reference, assigned seat number, and a unique check-in reference. It helps track which bookings have completed the check-in process and received seat allocation.

6. Repository Layer

Repositories abstract data access logic:

- **User Repository:** Supports fetching users by username or ID, adding new users, and updating existing ones.
- **Auth Repository:** Manages login validation and credential verification.
- **Admin Flight Repository:** Allows admins to add, update, or delete flights and manage flight schedules.

- **Flight Repository:** Enables flight search by origin, destination, and date; supports adding, updating, and deleting flights.
- **Booking Repository:** Handles booking creation, retrieval by reference number, fetching bookings with flight details, and updating bookings.
- **Check-In Repository:** Updates check-in status, assigns seat numbers, and records check-in dates.
- **Delete Repository:** Removes flights or bookings by ID with validation to ensure safe deletion.

7. Service Layer

Services encapsulate business logic:

- **Authentication Service:** Validates user credentials, checks username availability, and registers new users.
- **Booking Service:** Creates bookings by validating flight existence, generating a unique reference, and saving the booking. Also retrieves bookings by reference.
- **Admin Service:** Validates admin credentials (hardcoded), creates and updates flights, deletes flights, and updates user details.
- **Check-in Service:** Searches bookings and performs check-ins by validating conditions, generating a seat number, and updating the booking.
- **Flight Service:** Searches for flights based on origin, destination, and date.

8. Controller Endpoints

Controllers expose REST API endpoints for authentication, search, booking, check-in, and administration.

Authentication Endpoints

POST /api/Auth/register

: Registers a new passenger account after validating username and email uniqueness.

POST /api/Auth/login

: Validates credentials and returns a JWT bearer token.

: Token expiry is role-based, with different duration for admin and passenger roles.

Search Endpoints

- **GET /api/Search/view-trains**

: Returns the list of trains with route station information and seat/fare details.

- **POST /api/Search/check-fare-plan**

: Calculates fare for a train segment using source and destination station codes.

Booking Endpoints

- **POST /api/Booking/reserve-ticket**

: Creates reservation records after route validation, seat availability check, and fare calculation.

: Supports group booking up to six passengers in one request.

- **POST /api/Booking/cancel-ticket**

: Cancels an existing reservation using booking reference and updates booking/check-in state.

Check-In Endpoints

- **POST /api/CheckIn/perform**

: Performs online check-in, assigns seat number, and returns check-in confirmation.

- **POST /api/CheckIn/status**

: Returns current booking/check-in status for a reservation reference.

Admin Endpoints

- **POST /api/admin/newtrain**

: Creates a new train with route stations.

- **PUT /api/admin/updatetrain/{id}**

: Updates train details and validates station-code mapping.

- **DELETE /api/admin/delete-train/{id}**

: Deletes train data for the specified train identifier.

- **GET /api/admin/dashboard/{trainId}**

: Returns real-time train-level dashboard metrics such as booking counts, check-ins, and revenue.

9. Data Transfer Objects (DTOs)

Request DTOs include

- User registration request
- User login request
- Train search and fare-plan request
- Ticket reservation request (including multi-passenger support)
- Ticket cancellation request
- Check-in request
- Train creation with route request
- Train update request
- Booking status lookup request

Response DTOs include

- Reservation response
: Contains train details, source/destination station codes, fare per ticket, total fare, user email, and ticket list with generated reservation references.
- Reserved ticket item response
: Contains reservation reference, passenger details, booking status, check-in status, and seat assignment state (pending before check-in).
- Check-in result response
: Contains reservation reference, passenger name, assigned seat, check-in reference, and train identifier.
- Check-in status response
: Contains reservation reference, passenger name, current status, check-in flag, seat state, and status message.
- Fare-plan response
: Contains train details, source/destination station codes, station names, and calculated segment fare.
- Train dashboard response
: Contains train capacity, seats left, total bookings, confirmed/cancelled/check-in counts, and total revenue.

10. Middleware: Global Exception Handler

This middleware centralizes error handling across the application.

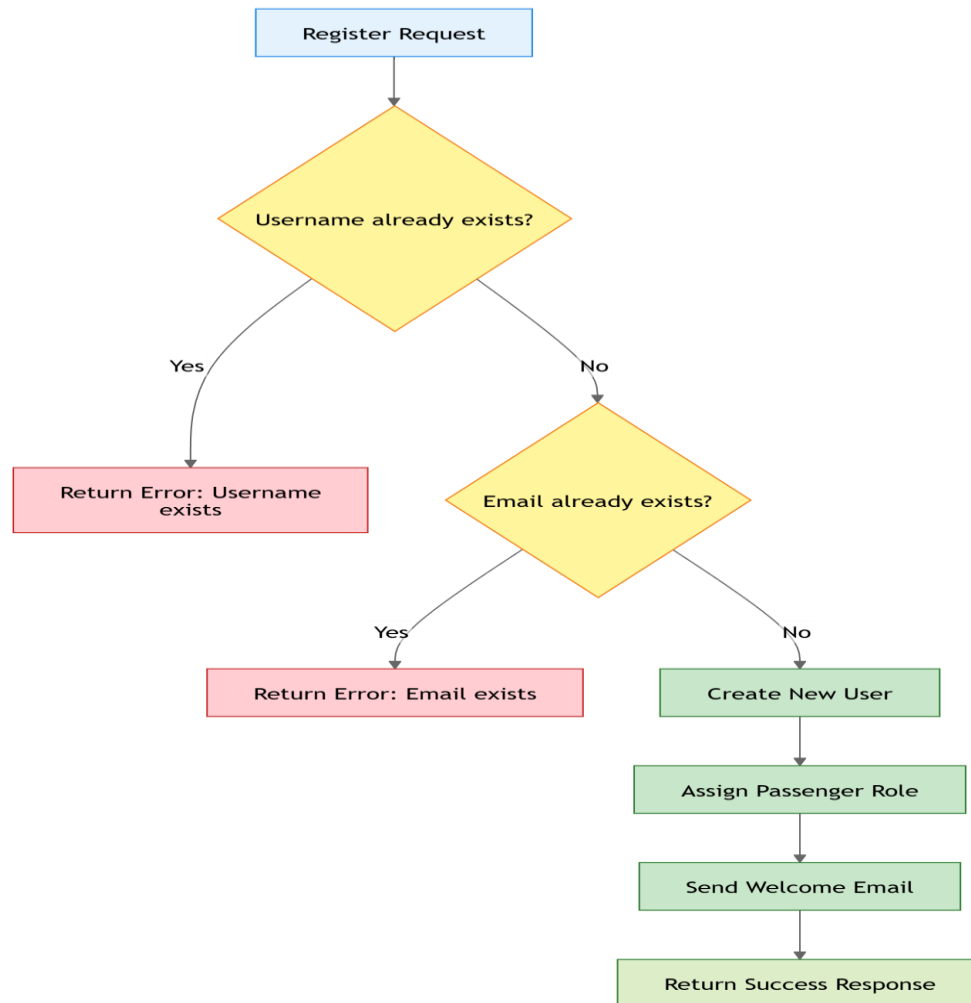
How it works

- Wraps each request in a try-catch pipeline.
- Intercepts unhandled exceptions from controllers, services, and repositories.
- Maps exceptions to appropriate HTTP status codes.
- Returns a structured JSON error response with message, status code, request path, trace identifier, and timestamp.

Exception Mapping

- Argument errors -> **400 Bad Request**
- Missing data / not found errors -> **404 Not Found**
- Unauthorized access errors -> **401 Unauthorized**
- Database update conflicts/errors -> **409 Conflict**
- Other unhandled errors -> **500 Internal Server Error**
- This approach ensures consistent, client-friendly error responses across all endpoints.

11. Key Functional Flows



User Registration

- The client sends a registration request.
- The system checks whether username and email already exist.
- If valid, it creates the passenger account, assigns the default passenger role, and returns a success response.
- A welcome email is also sent after successful registration.

User Login

- The client submits login credentials.
- The system validates the credentials and role claims.
- If valid, it generates a JWT bearer token and returns it in the response.
- Token expiry is role-aware for admin and passenger users.

Train Search and Fare Planning

- The client requests available trains and route details.
- The system returns train list, station routes, and seat/fare information.
- For fare planning, it validates source and destination station codes and calculates route-segment fare.

Ticket Booking

- The client selects a train and submits booking details.
- The system validates train availability, station-code direction, payment basics, and passenger count rules.
- It creates reservation records with unique references, updates seat inventory, and returns reservation details.
- Booking supports multiple passengers in one request (up to six).

Check-in Process

- The client submits a reservation reference for check-in.
- The system validates booking state and ensures it is eligible for check-in.
- It assigns the next available seat, stores check-in details, updates booking state, and returns confirmation.
- A check-in success email is sent after completion.

Cancellation Flow

- The client submits a reservation reference for cancellation.
- The system validates booking state, marks it cancelled, clears check-in/seat state, and updates train inventory as needed.
- A cancellation confirmation email is sent.

Admin Operations

- The admin logs in and receives an authorized token with admin privileges.
- Admin can create train routes, update train details, delete trains, and view dashboard metrics.
- All admin actions are validated and processed through the business service layer.

Exception Handling Flow

- Any request that triggers an error is intercepted by the global exception middleware.
- The middleware maps the exception to a suitable HTTP status code, builds a structured JSON payload, and returns a consistent error response to the client.

13. Security

The system uses JWT-based authentication and role-based authorization to secure API endpoints.

JWT Authentication

- Uses **HMAC-SHA256** for token signing.
- Tokens include claims such as username, token ID, and role information.
- Token expiry is role-aware:
 - : **Admin** tokens use shorter session policy.
 - : **Passenger** tokens use standard user session policy.

Token Generation

- User credentials are validated through the authentication service.
- Claims are created and signed using the configured secret key.
- Tokens include issuer, audience, expiration, and signing credentials.
- The final JWT is serialized and returned as a Bearer token.

Token Validation

- Token is extracted from the Authorization header.
- Signature is verified against the secret key.
- Expiration is validated.
- Issuer and audience settings are available in configuration for strict validation policy.

Authorization

- **Public endpoints:** registration, login, and open train search/fare APIs.
- **Protected endpoints:**
 - : Passenger role required for booking and check-in operations.
 - : Admin role required for train management and dashboard operations.

Current Security Posture

- Password handling is managed through ASP.NET Identity, which stores password hashes (not plain text).
- Admin account is bootstrapped at startup if missing, then managed through Identity roles.
- JWT middleware is integrated with authorization attributes for endpoint-level access control.

Security Recommendations for Production

- Enforce HTTPS-only deployment.
- Move JWT and SMTP secrets to secure secret storage (environment variables, key vault).
- Enable strict issuer and audience validation in JWT policy.
- Add refresh-token support with secure rotation.
- Add rate limiting and login attempt throttling.
- Add audit logging for authentication, booking, and admin actions.
- Add optional MFA for admin accounts.
- Harden CORS policy based on trusted client origins only.

14. Error Handling

Validation and error handling are layered across the application.

Validation Layers

- **Model-level validation** is applied through DTO structure and request contracts.
- **Controller-level validation** handles basic checks such as missing input, null request body, and required field presence.
- **Service-level validation** enforces domain rules such as route direction checks, booking limits, seat constraints, and status transitions.

Exception Strategy

- Services throw specific exceptions for invalid operations, missing resources, or business-rule violations.
- Typical exceptions include argument-related errors, not-found errors, unauthorized-access errors, and invalid-operation errors.
- All unhandled exceptions are captured by the global exception middleware.
- Middleware maps each exception to an appropriate HTTP status code and returns a structured JSON response.

Standard Error Response Behavior

The error response includes consistent fields such as:

- human-readable message
- HTTP status code
- request path

- trace identifier
- timestamp

This approach keeps API responses predictable for clients and improves debugging and observability.

15. Configuration

The application is configured using environment-based settings and startup dependency registration.

Configuration Settings

Primary settings include:

- SQL Server connection string
- JWT secret, issuer, and audience values
- logging preferences
- SMTP email settings for notification workflows

Dependency Registration

Startup registers:

- repository interfaces and implementations
- service interfaces and implementations
- controller services
- Swagger/OpenAPI services
- database context
- JWT bearer authentication and authorization

Middleware Pipeline

Request execution order is configured as:

1. global exception handling
2. authentication
3. authorization

4. controller endpoint mapping

This order ensures secure and consistent request processing.

Database Initialization

On startup, database initialization logic ensures baseline readiness:

- database creation check
- role initialization
- admin bootstrap initialization
- seed data loading for trains and routes when required

16. API Request and Response Examples

User Registration

Request: username, email, full name, and password

Success: user registration success message

Error: username already exists or email already registered

User Login

Request: username and password

Success: JWT bearer token

Error: invalid username or password

Train Search

Request: get available trains (and route station details)

Success: list of trains with station information, seats, and fare metadata

Fare Planning

Request: train identifier, source station code, destination station code

Success: calculated station-to-station fare for the selected route segment

Error: invalid station code or invalid route direction

Booking Creation (JWT required, Passenger role)

Request: train identifier, source/destination station code, passenger details, payment details

Success: reservation response with generated booking references, passenger

count, fare per ticket, total fare, and booking status

Error: train not found, insufficient seats, invalid station codes, invalid direction, or booking limit exceeded

Check-in (JWT required, Passenger role)

Request: booking reference

Success: check-in confirmation with assigned seat and check-in reference

Error: booking not found, already checked in, cancelled booking, or no seat available for assignment

Booking Cancellation (JWT required, Passenger role)

Request: booking reference

Success: cancellation confirmation message

Error: invalid booking reference or already-cancelled booking

Admin Train Creation (JWT required, Admin role)

Request: train details and full route station list

Success: train and route created successfully

Error: duplicate train identifier/number, invalid station data, or invalid request payload

Admin Train Update (JWT required, Admin role)

Request: train update details including source/destination station codes and optional intermediate station codes

Success: updated train response with normalized route mapping

Error: invalid station codes or route/body mismatch

Admin Dashboard (JWT required, Admin role)

Request: train identifier

Success: live capacity, booking status counts, check-in count, and revenue metrics

Error: train not found

Testing

The project uses unit and integration-style testing to validate API behavior and business logic.

Testing Stack

NUnit for writing and running test cases

Moq for mocking service/repository dependencies

EF Core InMemory for simulating data-layer behavior without real SQL Server

Testing Approach

- Controller tests verify HTTP responses and status codes.
- Service tests verify domain rules and workflow decisions.
- Data behavior is simulated using in-memory context to isolate logic.
- Mocking is used to replace external dependencies (repositories, email operations, identity-related dependencies) and test components in isolation.
- **Example Coverage Areas**
 - booking success and validation failures
 - check-in success and missing/invalid booking handling
 - authentication success/failure behavior
 - search and fare-plan response behavior
 - admin route operation behavior

Summary

Architecture: strict 3-tier layered system using controllers, services, repositories, and SQL Server.

Database: includes identity users, trains, route stations, bookings, and check-ins with constraints and indexes.

API: provides secured REST endpoints for authentication, search, booking, check-in, cancellation, and admin operations.

Design: uses repository pattern, dependency injection, DTO contracts, middleware pipeline, and service-layer business orchestration.

Security: JWT-based authentication with role-based authorization and token expiry policy.

Error Handling: centralized through global middleware with consistent HTTP status mapping and structured JSON errors.

Testing: implemented with NUnit, Moq, and EF Core InMemory for reliable validation of logic and endpoint behavior.